**Universidad Peruana de Ciencias Aplicadas**
**Facultad de Ingeniería**



**Curso:** Matemática Discreta

**Profesor:** Germain Cardenas Zavala

**Sección:** 3638

**Proyecto:** Cadenas de Markov

**Informe entregable de la semana 3**
"Aplicación de la Cadena de Markov en programación y en la vida real"

| **Integrantes** |
| --- |
| Chuquisengo Gomez, Aldighieri Farid |

Link del vídeo: https://youtu0.be/ZUwxuTU5shc

**2025-01**

# Índice

**1. Introducción** ..... **3**
**2. Objetivos** ..... **3**
2.1 Objetivos generales ..... 3
2.2 Objetivos específicos ..... 3
**3. Marco Teórico** ..... **3**
3.1. Vectores de probabilidad ..... 4
3.2. Matriz estocástica ..... 4
3.3. Cadena de Markov ..... 4
3.4. Teoría de Probabilidades ..... 5
**Reglas básicas del cálculo de probabilidades :** ..... **6**
**4. Ejemplos** ..... **7**
4.1. Ejemplo 1: Predicción simple del clima diario Contexto: ..... 7
4.2. Ejemplos 2 Predicción simple del clima con cadena de markov Contexto: ..... 8
**5. Diagramas : Diagrama de flujo** ..... **10**
**6. Código del Proyecto** ..... **11**
**7. Aplicación del Proyecto** ..... **13**
Resultados del Programa ..... 15
**8. Conclusiones y recomendaciones** ..... **17**
**9. Fuentes bibliográficas** ..... **18**

## 1. Introducción

Las cadenas de Markov constituyen un modelo matemático que permite describir la evolución de un sistema mediante transiciones probabilísticas entre distintos estados. Debido a su simplicidad y versatilidad, se han convertido en una herramienta relevante en diversas áreas del conocimiento, como la programación, el análisis predictivo y la toma de decisiones. Su aplicación permite estudiar el comportamiento de determinados sistemas, estimar posibles escenarios futuros y optimizar diferentes procesos.

En el presente trabajo, se analizará la aplicación de las cadenas de Markov mediante el estudio de una secuencia de estados climáticos. Para ello, se generará una secuencia aleatoria de entre 20 y 40 observaciones, considerando los siguientes estados:

- NU (nublado)
- P (parcialmente nublado)
- PS (parcialmente soleado)
- SO (soleado)

A partir de la secuencia, se contabilizarán las transiciones entre los estados climáticos y se construirá una matriz que represente su frecuencia, permitiendo analizar el comportamiento del sistema mediante las cadenas de Markov.

## 2. Objetivos

### 2.1 Objetivos generales

Profundizar el estudio de las Cadenas de Markov y su implementación en diversos ámbitos, como la programación y las aplicaciones del mundo real, mediante el uso del lenguaje de programación: Python.

### 2.2 Objetivos específicos

-Implementación del trabajo de programación

-Desarrollo del conocimiento de matemática discreta y aprender la implementación del curso dentro de la programación.

-Desarrollar y programar simulaciones que permitan observar y analizar el programa del problema propuesto.

-Construir una matriz estocástica que represente las probabilidades de transición entre diferentes estados climáticos simulados.

-Resolución del problema propuesto del proyecto: Modelo de Leslie en ecología.

## 3. Marco Teórico

### 3.1. Vectores de probabilidad

Un vector de probabilidades es un conjunto ordenado de números reales no negativos cuya suma total es igual a 1. Cada número del vector representa la probabilidad de que ocurra un determinado estado. Por ejemplo, el vector [0.6, 0.3, 0.1] podría representar un 60% de probabilidad de que el clima esté soleado, 30% de que esté parcialmente nublado y 10% de que llueva.

En el contexto de las cadenas de Markov, los vectores de probabilidad representan el estado del sistema en un instante determinado, indicando la probabilidad de estar en cada uno de los estados posibles.

### 3.2. Matriz estocástica

Una matriz estocástica es una matriz cuadrada en la que cada fila representa un estado actual, y cada columna un posible estado futuro. Los elementos de la matriz son probabilidades, por lo tanto, toman valores entre 0 y 1, y la suma de los elementos de cada fila es igual a 1.

Estas matrices permiten modelar las probabilidades de transición entre estados. En este proyecto, la matriz estocástica refleja las probabilidades de pasar de un tipo de clima a otro (por ejemplo, de nublado a soleado). Multiplicando un vector de probabilidad inicial por la matriz estocástica, se puede obtener la distribución de probabilidad para el siguiente día.

### 3.3. Cadena de Markov

Una cadena de Markov es un modelo matemático utilizado para describir un proceso que cambia de estado en forma secuencial, donde la probabilidad del siguiente estado depende únicamente del estado actual. Esta propiedad se conoce como propiedad de Markov.

En otras palabras, el futuro del sistema depende solo del presente, no del pasado. Por ejemplo: si hoy está soleado, y se sabe que hay un 70% de que mañana también lo esté, esa información es suficiente para predecir el clima, sin necesidad de analizar cómo estuvo el clima en días anteriores.

En este proyecto, se utiliza una cadena de Markov de tiempo discreto y estado finito para simular cómo cambia el clima a lo largo de varios días. A partir de una secuencia de estados aleatorios y del conteo de transiciones entre ellos, se construye la matriz estocástica. Esta matriz permite hacer predicciones basadas únicamente en el estado actual.

### 3.4. Teoría de Probabilidades

La teoría de probabilidades es una rama de la matemática que estudia la posibilidad de que ocurra un evento (Wolfram MathWorld). En términos simples, mide cuán probable es que ocurra un determinado resultado dentro de un conjunto de todos los posibles.Se expresa mediante un número entre 0 y 1:

- 0 significa que el evento es imposible.
- 1 significa que el evento es seguro.
- Valores entre 0 y 1 indican niveles de certeza intermedios. Ejemplo: Al lanzar una moneda justa, la probabilidad de que salga cara es 0.5, ya que hay dos resultados igualmente posibles (cara o cruz).

Modelos de probabilidad Existen varios enfoques teóricos sobre cómo entender y definir la probabilidad:

1. **Enfoque clásico:**
   - Define la probabilidad como el cociente entre los casos favorables y los casos posibles, suponiendo que todos son igualmente probables.

$$P(A) = \frac{número\ de\ casos\ favorables\ a\ A}{número\ \ total\ de\ casos\ favorables}$$

   - Ejemplo: Lanzar un dado. Probabilidad de que salga un 4:

$$P(4) = \frac{1}{6}$$

2. **Enfoque frecuentista:**
   - Define la probabilidad como el límite de la frecuencia relativa de un evento cuando el experimento se repite muchas veces.
   - Ejemplo: Si lanzamos una moneda 1,000 veces y sale cara 502 veces, la frecuencia relativa es 0.502.

3. **Enfoque bayesiano (subjetivo):**
   - Considera la probabilidad como una medida del grado de creencia o incertidumbre subjetiva respecto a un evento.
   - Se actualiza con nueva información mediante el teorema de Bayes.

Elementos básicos

- Experimento aleatorio: Proceso cuyo resultado no puede preverse con certeza (ej.: lanzar un dado).
- Espacio muestral (S): Conjunto de todos los posibles resultados.
- Evento (A): Subconjunto del espacio muestral (el resultado que nos interesa).

**Reglas básicas del cálculo de probabilidades :**

1. Probabilidad de un evento seguro:

$P(S) = 1$

2. Probabilidad de un evento imposible:

$P(\emptyset) = 0$

3. Complemento de un evento:

$P(no\ A) = 1 - P(A)$

4. Unión de dos eventos (si son mutuamente excluyentes):

$P(A \cup B) = P(A) + P(B)$

5. Intersección de dos eventos (si son independientes):

$P(A \cap B) = P(A) \cdot P(B)$

## Tipos de eventos

- Simples: Contienen un solo resultado del espacio muestral.
- Compuestos: Contienen dos o más resultados.
- Mutuamente excluyentes: No pueden ocurrir al mismo tiempo.
- Independientes: La ocurrencia de uno no afecta al otro.

## 4. Ejemplos

### 4.1. Ejemplo 1: Predicción simple del clima diario

Contexto**:**

Una ciudad tiene tres posibles condiciones climáticas diarias: SO = Soleado, NU = Nublado y LL = Lluvioso.

Probabilidades de transición:

- Desde SO: 70% SO, 20% NU, 10% LL

- Desde NU: 40% SO, 40% NU, 20% LL

- Desde LL: 30% SO, 30% NU, 40% LL

Matriz de transición P:

| De / A | SO | NU | LL |

|--------|------|------|

| SO | 0.70 | 0.20 | 0.10 |

| NU | 0.40 | 0.40 | 0.20 |

| LL | 0.30 | 0.30 | 0.40 |

Vector de estado inicial:

v0 = [1, 0, 0] (100% probabilidad de SO)

Día 1:

v1 = v0 * P = [0.70, 0.20, 0.10]

Día 2:

v2 = v1 * P = [0.60, 0.25, 0.15]

Día 3:

v3 = v2 * P = [0.565, 0.265, 0.17]

Resultado:

Día 1: 70% SO, 20% NU, 10% LL

Día 2: 60% SO, 25% NU, 15% LL

Día 3: 56.5% SO, 26.5% NU, 17% LL

Se usa una cadena de Markov, donde el clima de un día depende solo del día anterior. La predicción se hace multiplicando el vector de estado actual por una matriz de transición de probabilidades para obtener la probabilidad del clima del día siguiente.

### 4.2. Ejemplos 2 Predicción simple del clima con cadena de

### markov

Contexto**:**

Una ciudad tiene tres posibles condiciones climáticas diarias: soleado, nublado y lluvioso.
Se ha observado históricamente que si hoy está soleado, hay un 70% de probabilidad de que mañana también lo esté, 20% de que esté nublado y 10% de que llueva.

Aplicación de la cadena de Markov:
Se construye una matriz de transición con estas probabilidades y se utiliza para predecir el clima de los días siguientes, basándose únicamente en el clima de hoy. Esto permite generar pronósticos a corto plazo sin necesidad de analizar todos los días anteriores, solo el estado actual.

Predicción del Clima con Cadenas de Márkov
Supuesto:
Se observaron 10 días de clima con los siguientes estados:
["SO", "SO", "PS", "NU", "P", "P", "NU", "SO", "PS", "SO"]

Paso 1: Contar transiciones entre estados
Estados: SO = Soleado, PS = Parcialmente Soleado, NU = Nublado, P = Precipitaciones

Transiciones:
| De -> A | SO | PS | NU | P |
|--------|----|----|----|---|
| SO | 1 | 1 | 0 | 0 |
| PS | 0 | 0 | 1 | 0 |
| NU | 1 | 0 | 0 | 1 |
| P | 0 | 0 | 1 | 1 |

Paso 2: Matriz Estocástica (normalizada por fila):
| De / A | SO | PS | NU | P |
|--------|------|------|------|  |
| SO | 0.5 | 0.5 | 0.0 | 0.0 |
| PS | 0.0 | 0.0 | 1.0 | 0.0 |
| NU | 0.5 | 0.0 | 0.0 | 0.5 |
| P | 0.0 | 0.0 | 0.5 | 0.5 |

Paso 3: Predicción desde estado PS

Día 1: v1 = [0.0, 0.0, 1.0, 0.0] -> NU (100%)
Día 2: v2 = [0.5, 0.0, 0.0, 0.5] -> 50% SO, 50% P -> elegimos P
Día 3: v3 = [0.0, 0.0, 0.5, 0.5] -> 50% NU, 50% P -> elegimos NU

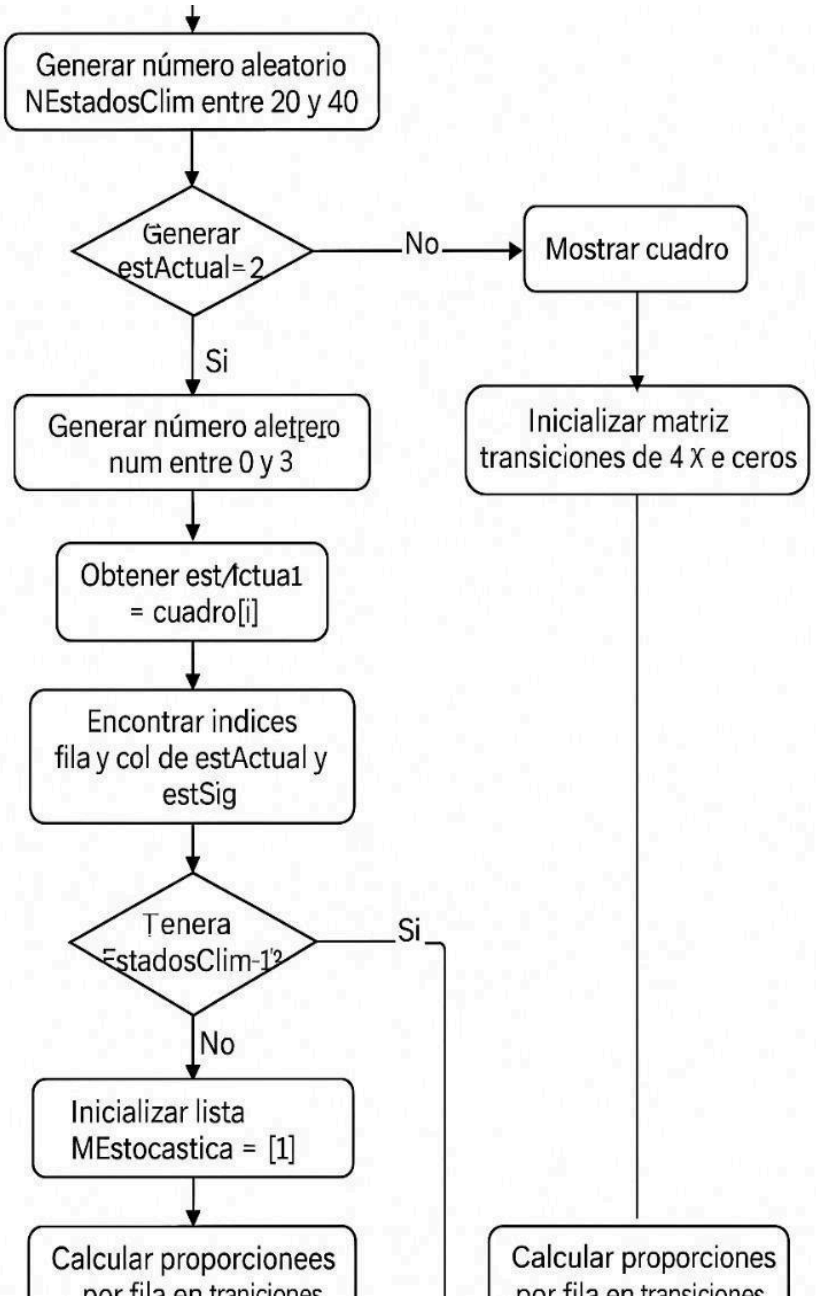
Resultado:

- Día 1: NU
- Día 2: P
- Día 3: NU

Resultado:
- Día 1: NU
- Día 2: P
- Día 3: NU

Criterios:
- Se escoge el estado con mayor probabilidad.
- En caso de empate, se selecciona aleatoriamente o se indica la ambigüedad.

## 5. Diagramas : Diagrama de flujo



## 6. Código del Proyecto

```
import random
import tkinter as tk
from tkinter import ttk


estadosPosibles = ["NU", "P", "PS", "SO"]

MATRIZ_MODELO = [
    [0.00, 0.67, 0.33, 0.00],
    [0.00, 0.40, 0.60, 0.00],
    [0.00, 0.00, 0.10, 0.90],
    [0.40, 0.20, 0.00, 0.40],
]

COLORES_ESTADOS = {
    "NU": "#64748b",
    "P": "#38bdf8",
    "PS": "#fbbf24",
    "SO": "#fb7185",
}

FONDO = "#f1f5f9"
TEXTO = "#0f172a"
TEXTO_SECUNDARIO = "#64748b"
AZUL = "#2563eb"


def generarCuadro(n, estado_inicial=None):
    estado = estado_inicial or random.choice(estadosPosibles)
    cuadro = [estado]
    for _ in range(n - 1):
        indice = estadosPosibles.index(estado)
        estado = random.choices(
            estadosPosibles, weights=MATRIZ_MODELO[indice], k=1
        )[0]
        cuadro.append(estado)
    return cuadro
```

```
def actualizarMatriz(matriz, estado_anterior, estado_actual):
    if estado_anterior is not None:
        i = estadosPosibles.index(estado_anterior)
        j = estadosPosibles.index(estado_actual)
        matriz[i][j] += 1
    return matriz


def calcularMatrizEstocastica(transiciones):
    matrizEstocastica = []
    for fila in transiciones:
        total = sum(fila)
        nuevaFila = []
        for x in fila:
            nuevaFila.append(round(x / total, 2) if total else 0.0)
        matrizEstocastica.append(nuevaFila)
    return matrizEstocastica


class DashboardMarkov:
    def __init__(self, root):
        self.root = root
        self.root.title("Dashboard | Cadenas de Markov")
        self.root.geometry("1100x760")
        self.root.minsize(860, 620)
        self.root.configure(bg=FONDO)

        self.cuadro = []
        self.transiciones = [[0] * len(estadosPosibles) for _ in
estadosPosibles]

        self._configurar_estilos()
        self._crear_interfaz()

    def _configurar_estilos(self):
        estilo = ttk.Style()
        estilo.theme_use("clam")
        estilo.configure(
            "TButton",
            font=("Segoe UI", 10, "bold"),
```

```
            padding=(14, 9),
            background=AZUL,
            foreground="white",
            borderwidth=0,
        )
        estilo.map("TButton", background=[("active", "#1d4ed8")])
        estilo.configure(
            "TSpinbox",
            font=("Segoe UI", 11),
            padding=8,
            fieldbackground="white",
        )

    def _crear_interfaz(self):
        encabezado = tk.Frame(self.root, bg="#0f172a", padx=28, pady=20)
        encabezado.pack(fill="x")

        tk.Label(
            encabezado,
            text="CADENAS DE MARKOV",
            bg="#0f172a",
            fg="white",
            font=("Segoe UI", 19, "bold"),
        ).pack(anchor="w")
        tk.Label(
            encabezado,
            text="Dashboard de simulación de estados climáticos",
            bg="#0f172a",
            fg="#cbd5e1",
            font=("Segoe UI", 10),
        ).pack(anchor="w", pady=(4, 0))

        contenido = tk.Frame(self.root, bg=FONDO, padx=24, pady=20)
        contenido.pack(fill="both", expand=True)

        configuracion = tk.Frame(contenido, bg="white", padx=18, pady=14)
        configuracion.pack(fill="x", pady=(0, 16))
        tk.Label(
            configuracion,
            text="Estados iniciales (20-40)",
            bg="white",
            fg=TEXTO,
```

```
            font=("Segoe UI", 10, "bold"),
        ).pack(side="left")
        self.cantidad_var = tk.IntVar(value=20)
        ttk.Spinbox(
            configuracion,
            from_=20,
            to=40,
            width=5,
            textvariable=self.cantidad_var,
            style="TSpinbox",
        ).pack(side="left", padx=(12, 14))
        tk.Label(
            configuracion,
            text="Estado inicial",
            bg="white",
            fg=TEXTO,
            font=("Segoe UI", 10, "bold"),
        ).pack(side="left")
        self.estado_inicial_var = tk.StringVar(value="Aleatorio")
        ttk.Combobox(
            configuracion,
            textvariable=self.estado_inicial_var,
            values=["Aleatorio", *estadosPosibles],
            state="readonly",
            width=11,
        ).pack(side="left", padx=(10, 14))
        ttk.Button(
            configuracion,
            text="Iniciar / reiniciar simulación",
            command=self.iniciar_simulacion,
        ).pack(side="left")
        ttk.Button(
            configuracion,
            text="+ Agregar estado",
            command=self.agregar_estado,
        ).pack(side="right")

        self.error_var = tk.StringVar()
        tk.Label(
            contenido,
            textvariable=self.error_var,
            bg=FONDO,
```

```
            fg="#dc2626",
            font=("Segoe UI", 9),
        ).pack(anchor="w", pady=(0, 6))

        tarjetas = tk.Frame(contenido, bg=FONDO)
        tarjetas.pack(fill="x", pady=(0, 16))
        self.total_var = tk.StringVar(value="—")
        self.dia_var = tk.StringVar(value="—")
        self.estado_var = tk.StringVar(value="—")
        self._crear_tarjeta(tarjetas, "TOTAL DE ESTADOS", self.total_var,
"#2563eb")
        self._crear_tarjeta(tarjetas, "DÍA ACTUAL", self.dia_var, "#7c3aed")
        self._crear_tarjeta(tarjetas, "ESTADO ACTUAL", self.estado_var,
"#db2777")

        paneles = tk.Frame(contenido, bg=FONDO)
        paneles.pack(fill="both", expand=True)
        paneles.columnconfigure(0, weight=3, uniform="panel")
        paneles.columnconfigure(1, weight=2, uniform="panel")
        paneles.rowconfigure(0, weight=1)

        historial = tk.Frame(paneles, bg="white", padx=18, pady=16)
        historial.grid(row=0, column=0, sticky="nsew", padx=(0, 8))
        tk.Label(
            historial,
            text="Historial de estados",
            bg="white",
            fg=TEXTO,
            font=("Segoe UI", 13, "bold"),
        ).pack(anchor="w")
        tk.Label(
            historial,
            text="Los días más recientes aparecen primero",
            bg="white",
            fg=TEXTO_SECUNDARIO,
            font=("Segoe UI", 9),
        ).pack(anchor="w", pady=(3, 12))

        area_historial = tk.Frame(historial, bg="white")
        area_historial.pack(fill="both", expand=True)
        self.historial_canvas = tk.Canvas(
            area_historial, bg="white", highlightthickness=0
```

```
        )
        scrollbar = ttk.Scrollbar(
            area_historial, orient="vertical",
command=self.historial_canvas.yview
        )
        self.historial_canvas.configure(yscrollcommand=scrollbar.set)
        self.historial_canvas.pack(side="left", fill="both", expand=True)
        scrollbar.pack(side="right", fill="y")
        self.historial_frame = tk.Frame(self.historial_canvas, bg="white")
        self.historial_window = self.historial_canvas.create_window(
            (0, 0), window=self.historial_frame, anchor="nw"
        )
        self.historial_frame.bind(
            "<Configure>",
            lambda _event: self.historial_canvas.configure(
                scrollregion=self.historial_canvas.bbox("all")
            ),
        )
        self.historial_canvas.bind(
            "<Configure>",
            lambda event: self.historial_canvas.itemconfigure(
                self.historial_window, width=event.width
            ),
        )

        matriz_panel = tk.Frame(paneles, bg="white", padx=18, pady=16)
        matriz_panel.grid(row=0, column=1, sticky="nsew", padx=(8, 0))
        tk.Label(
            matriz_panel,
            text="Matriz estocástica",
            bg="white",
            fg=TEXTO,
            font=("Segoe UI", 13, "bold"),
        ).pack(anchor="w")
        tk.Label(
            matriz_panel,
            text="Probabilidad de transición entre estados",
            bg="white",
            fg=TEXTO_SECUNDARIO,
            font=("Segoe UI", 9),
        ).pack(anchor="w", pady=(3, 14))
```

```
        pestanas = ttk.Notebook(matriz_panel)
        pestanas.pack(fill="both", expand=True)
        self.tablas = {
            "frecuencias": self._crear_tabla_matriz(pestanas,
"Frecuencias"),
            "observada": self._crear_tabla_matriz(
                pestanas, "Prob. observadas"
            ),
            "modelo": self._crear_tabla_matriz(pestanas, "Modelo"),
        }
        for estado, color in COLORES_ESTADOS.items():
            tk.Label(
                matriz_panel,
                text=f"●  {estado}",
                bg="white",
                fg=color,
                font=("Segoe UI", 9, "bold"),
            ).pack(anchor="w", pady=(12 if estado == "NU" else 3, 0))

        tk.Label(
            matriz_panel,
            text="El siguiente estado se sortea según la fila del modelo
correspondiente al estado actual.",
            bg="white",
            fg=TEXTO_SECUNDARIO,
            font=("Segoe UI", 8),
            wraplength=320,
            justify="left",
        ).pack(anchor="w", pady=(12, 0))

        self.iniciar_simulacion()

    def _crear_tabla_matriz(self, parent, titulo):
        marco = tk.Frame(parent, bg="white")
        parent.add(marco, text=titulo)
        tabla = ttk.Treeview(
            marco,
            columns=("origen", *estadosPosibles),
            show="headings",
            height=5,
        )
        tabla.heading("origen", text="De / A")
```

```
        tabla.column("origen", width=55, minwidth=50, anchor="center")
        for estado in estadosPosibles:
            tabla.heading(estado, text=estado)
            tabla.column(estado, width=48, minwidth=40, anchor="center")
        tabla.pack(fill="both", expand=True, padx=4, pady=4)
        return tabla

    def _crear_tarjeta(self, parent, titulo, variable, color):
        tarjeta = tk.Frame(parent, bg="white", padx=18, pady=14)
        tarjeta.pack(side="left", fill="x", expand=True, padx=6)
        tk.Frame(tarjeta, bg=color, width=4).pack(side="left", fill="y",
padx=(0, 12))
        textos = tk.Frame(tarjeta, bg="white")
        textos.pack(side="left", fill="x", expand=True)
        tk.Label(
            textos,
            text=titulo,
            bg="white",
            fg=TEXTO_SECUNDARIO,
            font=("Segoe UI", 8, "bold"),
        ).pack(anchor="w")
        tk.Label(
            textos,
            textvariable=variable,
            bg="white",
            fg=TEXTO,
            font=("Segoe UI", 20, "bold"),
        ).pack(anchor="w", pady=(3, 0))

    def iniciar_simulacion(self):
        try:
            cantidad = self.cantidad_var.get()
        except (tk.TclError, ValueError):
            self.error_var.set("Ingrese un número entero entre 20 y 40.")
            return

        if not 20 <= cantidad <= 40:
            self.error_var.set("La cantidad debe estar entre 20 y 40.")
            return

        self.error_var.set("")
        estado_inicial = self.estado_inicial_var.get()
```

```
        if estado_inicial == "Aleatorio":
            estado_inicial = None
        self.cuadro = generarCuadro(cantidad, estado_inicial)
        self.transiciones = [[0] * len(estadosPosibles) for _ in
estadosPosibles]
        for anterior, actual in zip(self.cuadro, self.cuadro[1:]):
            actualizarMatriz(self.transiciones, anterior, actual)
        self.actualizar_dashboard()

    def agregar_estado(self):
        if not self.cuadro:
            self.error_var.set("Inicie primero una simulación.")
            return

        anterior = self.cuadro[-1]
        fila_modelo = MATRIZ_MODELO[estadosPosibles.index(anterior)]
        nuevo_estado = random.choices(
            estadosPosibles, weights=fila_modelo, k=1
        )[0]
        self.cuadro.append(nuevo_estado)
        actualizarMatriz(self.transiciones, anterior, nuevo_estado)
        self.error_var.set("")
        self.actualizar_dashboard()

    def actualizar_dashboard(self):
        self.total_var.set(str(len(self.cuadro)))
        self.dia_var.set(str(len(self.cuadro)))
        self.estado_var.set(self.cuadro[-1] if self.cuadro else "—")

        for widget in self.historial_frame.winfo_children():
            widget.destroy()

        for indice, estado in enumerate(reversed(self.cuadro)):
            dia = len(self.cuadro) - indice
            nombre_dia = "HOY" if indice == 0 else f"DÍA {dia}"
            tarjeta = tk.Frame(
                self.historial_frame,
                bg="#f8fafc",
                padx=10,
                pady=9,
                highlightbackground="#e2e8f0",
                highlightthickness=1,
```

```
            )
            tarjeta.grid(
                row=indice // 3,
                column=indice % 3,
                sticky="ew",
                padx=4,
                pady=4,
            )
            tk.Label(
                tarjeta,
                text=nombre_dia,
                bg="#f8fafc",
                fg=TEXTO_SECUNDARIO,
                font=("Segoe UI", 8, "bold"),
            ).pack(anchor="w")
            tk.Label(
                tarjeta,
                text=estado,
                bg="#f8fafc",
                fg=COLORES_ESTADOS[estado],
                font=("Segoe UI", 15, "bold"),
            ).pack(anchor="w", pady=(2, 0))

        for columna in range(3):
            self.historial_frame.columnconfigure(columna, weight=1)
        self.historial_canvas.yview_moveto(0)

        probabilidades = calcularMatrizEstocastica(self.transiciones)
        self._actualizar_tabla("frecuencias", self.transiciones,
decimales=0)
        self._actualizar_tabla("observada", probabilidades)
        self._actualizar_tabla("modelo", MATRIZ_MODELO)

    def _actualizar_tabla(self, nombre, matriz, decimales=2):
        tabla = self.tablas[nombre]
        for item in tabla.get_children():
            tabla.delete(item)
        for estado, fila in zip(estadosPosibles, matriz):
            if decimales == 0:
                valores = tuple(str(int(valor)) for valor in fila)
            else:
                valores = tuple(f"{valor:.{decimales}f}" for valor in fila)
```

```
            tabla.insert("", "end", values=(estado, *valores))


def main():
    Dash = tk.Tk()
    DashboardMarkov(Dash)
    Dash.mainloop()


if __name__ == "__main__":
    main()
```

## 7. Aplicación del Proyecto

1) **El programa genera la siguiente secuencia de estados climáticos para 20 números: [PS, P, PS, SO, SO, NU, NU, P, P, PS, SO, SO, NU, PS, P, PS, SO, P, PS, SO]**

   **El programa analiza cuántas veces cambia un estado a otro. Por ejemplo:**

- De `PS` a `SO` ocurrió 3 veces.
- De `P` a `PS` ocurrió 2 veces.
- De `NU` a `P` ocurrió 1 vez.

Se construye una matriz con las **frecuencias** de transiciones entre cada estado.

| | **NU** | **P** | **PS** | **SO** |
|---|---|---|---|---|
| **NU** | **0.00** | **0.50** | **0.50** | **0.00** |
| **P** | **0.00** | **0.25** | **0.50** | **0.25** |
| **PS** | **0.00** | **0.00** | **0.25** | **0.75** |
| **SO** | **0.25** | **0.00** | **0.25** | **0.50** |

2) **Supongamos que el usuario ingresa un historial de 25 días y el programa genera aleatoriamente la siguiente secuencia climática:**

   **[NU, P, P, PS, SO, NU, PS, PS, SO, SO, P, P, NU, NU, PS, SO, P, P, PS, SO, SO, NU, P, PS, SO]**

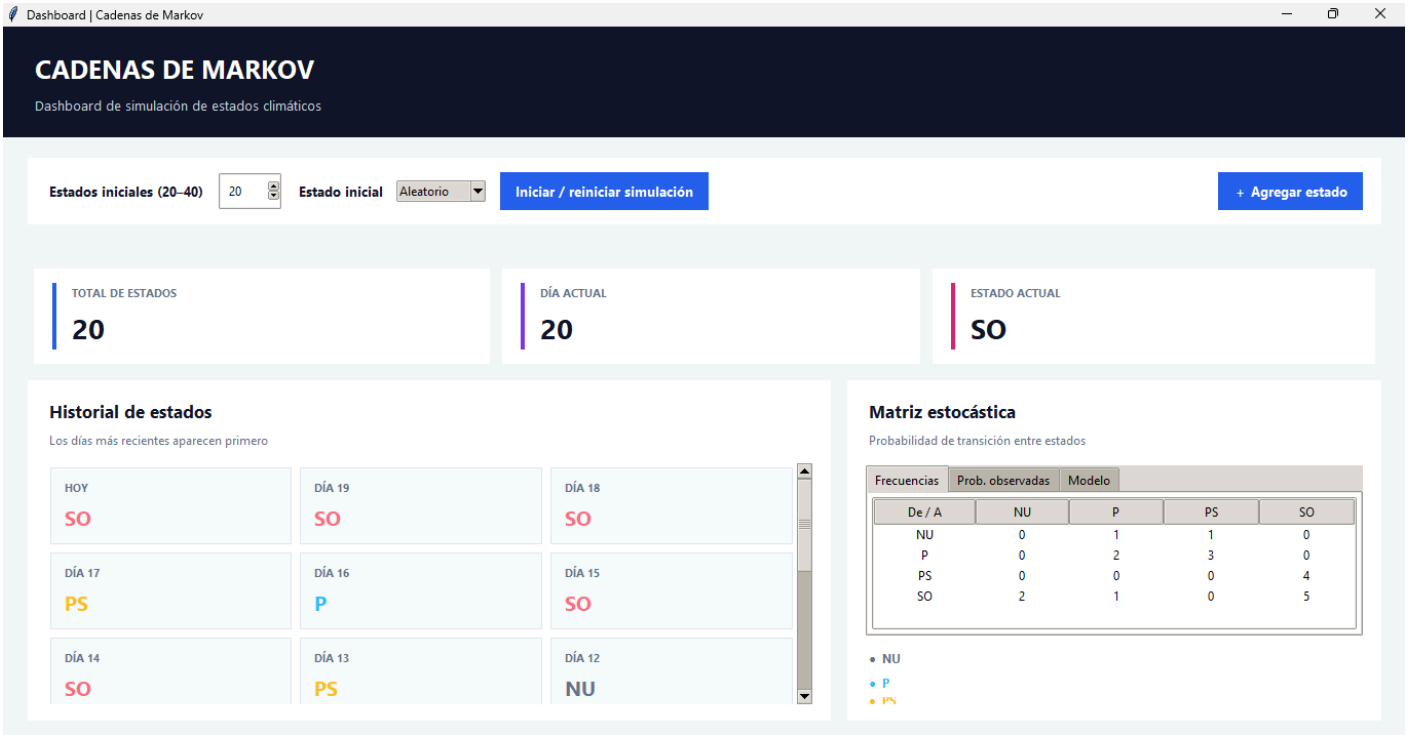
**El programa analiza cuántas veces ocurre cada cambio entre un estado y otro:**

- De NU a P: 2 veces
- De P a P: 2 veces
- De P a PS: 3 veces
- De PS a SO: 4 veces
- De SO a NU: 2 veces
- De SO a P: 1 vez

**Estos datos se usan para construir la matriz de frecuencias de transición.**

| | NU | P | PS | SO |
|---|---|---|---|---|
| NU | 0.00 | 0.67 | 0.33 | 0.00 |
| P | 0.00 | 0.40 | 0.60 | 0.00 |
| PS | 0.00 | 0.00 | 0.10 | 0.90 |
| SO | 0.40 | 0.20 | 0.00 | 0.40 |

## Resultados del Programa



## 8. Conclusiones y recomendaciones

El modelo implementado representa una herramienta didáctica efectiva para ilustrar cómo las cadenas de Markov pueden capturar transiciones entre estados discretos, como en el caso de condiciones climáticas. A través de este ejercicio, se evidenció que:

- Aunque el modelo cumple su propósito pedagógico, su aplicación en escenarios reales requeriría una mayor granularidad en los estados, el uso de datos históricos reales en lugar de valores aleatorios, y posibles extensiones como los Modelos Ocultos de Markov para representar factores no observables directamente.

- Se recomienda incorporar técnicas de suavizado, como el suavizado de Laplace, para evitar transiciones con probabilidad cero, especialmente cuando se trabaja con conjuntos de datos pequeños.

- La inclusión de condiciones climáticas adicionales (como “tormenta” o “nieve”) podría enriquecer el modelo y hacerlo más representativo de la realidad.

- Finalmente, se sugiere evaluar si la matriz de transición converge hacia una distribución estacionaria, lo cual indicaría la existencia de un equilibrio climático teórico.

Estos resultados y reflexiones permiten comprender tanto las fortalezas como las limitaciones del modelo, sentando las bases para futuros trabajos más robustos y orientados a aplicaciones reales.

## 9. Fuentes bibliográficas

-Jimenez, E. (2019). Cadenas de Markov espaciales para simular el crecimiento del Área Metropolitana de Toluca, 2017-2031. *Economía, sociedad y territorio. 19(60).* https://www.scielo.org.mx/scielo.php?pid=S1405-84212019000200109&script=sci_arttext# B34

-Scientia et Technica (2006). *Convergencia de LAS CADENAS DE Markov* https://revistas.utp.edu.co/index.php/revistaciencia/article/view/6195/3605

-Barriola, J. & Dotta, M. *¿Cómo funciona Google? El algoritmo pagerank, diagramas de grafos y cadenas de markov. Revista de Investigación en Modelos Matemáticos Aplicados a la Gestión y la Economía. 3. 9-30.* https://www.economicas.uba.ar/wp-content/uploads/2016/04/2-%C2%BFC%C3%B3mo-fun
ciona-google_-el-algoritmo-pagerank-diagramas-de-grafos-y-cadenas-de-markov.-Juan-Manuel-Barriola-y-Milena-Dotta-2.pdf

- Weisstein, E. W. (s.f.). *Probability*. Wolfram MathWorld. Recuperado el 22 de mayo de 2025, de https://mathworld.wolfram.com/Probability.html

-